

Tarefa 15: Comunicação Não Bloqueante em MPI

Difusão de calor em uma barra 1D com sobreposição de computação e comunicação

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Setembro de 2026

1. Objetivo

Implementar uma simulação da difusão de calor em uma barra 1D, dividida entre dois ou mais processos MPI, em que cada processo simula um trecho da barra com células extras para a troca de bordas com os vizinhos. Implementar três versões — com MPI_Send/MPI_Recv, com MPI_Isend/MPI_Irecv e MPI_Wait, e com MPI_Test para atualizar os pontos internos enquanto a comunicação não termina — comparar os tempos de execução e discutir os ganhos com a sobreposição de comunicação e computação.

2. Ambiente de Execução

Item	Valor
Máquina	Supercomputador NPAD/UFRN, partição amd-512, nós exclusivos
Nós	r2n01 e r2n02 — 16 processos, 8 por nó, distribuídos alternadamente (- -map-by node)
Processador	2 × AMD EPYC 7713 (Milan) por nó
MPI	OpenMPI 5.0.10 (libraries/openmpi/5.0.10-gnu14-ucxmt)
Compilador	GCC 14.2.0 via mpicc -O2
Job	SLURM 2127428

3. Implementação

A barra é dividida em trechos de n pontos, um por processo. Cada processo guarda seu trecho em $u[1..n]$ e duas células extras, $u[0]$ e $u[n+1]$, que recebem o valor da borda dos vizinhos. A cada passo, o novo valor de cada ponto é $v[i] = u[i] + K \cdot (u[i-1] - 2u[i] + u[i+1]))$, com $K = 0,25$ (estável). A ponta esquerda da barra é mantida em 100 e a direita em 0; nas pontas, a célula extra não é trocada e guarda esse valor.

Versão	Troca de bordas	Atualização
v1	MPI_Send/MPI_Recv. Processos pares enviam enquanto ímpares recebem e depois invertem, para não haver impasse	Depois da troca, todos os pontos
v2	MPI_Irecv e MPI_Isend com os dois vizinhos, guardando um MPI_Request por operação	Pontos internos (2 a $n-1$), MPI_Wait nas requisições e, por fim, os pontos 1 e n
v3	Igual à v2	Pontos internos em blocos de 256, chamando MPI_Test entre os blocos até a comunicação terminar; depois os pontos 1 e n

Os pontos internos não dependem das células extras, então podem ser calculados enquanto as mensagens estão em trânsito; só os pontos 1 e n precisam esperar. As três versões produzem o mesmo resultado (o valor final de $u[n]$ no processo 0 é idêntico).

O trabalho total foi mantido fixo ($n \times \text{passos} = 10^9$) e n variou de 100 a 1 000 000 pontos por processo. Os processos vizinhos ficam em nós diferentes, então toda troca de borda passa pela rede. Cada caso

rodou 3 vezes e foi usado o menor tempo, medido com MPI_Wtime no processo 0.

4. Resultados

Pontos por processo	Passos	v1 ($\mu\text{s}/\text{passo}$)	v2 ($\mu\text{s}/\text{passo}$)	v3 ($\mu\text{s}/\text{passo}$)	Ganho v2	Ganho v3
100	10.000.000	2,86	1,46	1,48	1,96×	1,93×
1.000	1.000.000	3,74	2,01	1,98	1,86×	1,89×
10.000	100.000	11,42	9,37	10,16	1,22×	1,12×
100.000	10.000	112,85	87,48	93,20	1,29×	1,21×
1.000.000	1.000	2.955,28	2.995,27	3.206,05	0,99×	0,92×

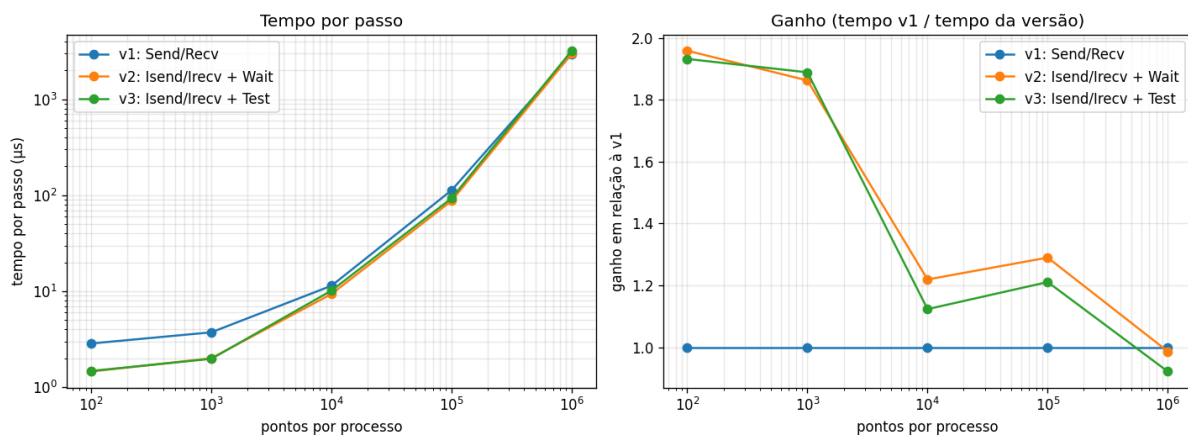


Figura 1 — Tempo por passo (esquerda) e ganho em relação à versão bloqueante (direita) em função do número de pontos por processo.

5. Análise

Trechos pequenos (100 pontos): a comunicação domina. A conta leva poucas dezenas de nanossegundos e o passo custa praticamente só a troca de bordas. A v1 leva 2,9 μs por passo e as versões não bloqueantes, 1,5 μs : metade. Com MPI_Send/MPI_Recv as trocas acontecem em duas fases (pares enviam, depois ímpares), ou seja, duas latências de rede por passo; com MPI_Isend/MPI_Irecv as quatro mensagens de cada processo ficam em trânsito ao mesmo tempo e o passo paga uma latência só. Aqui o ganho vem de não serializar a comunicação, não da sobreposição, porque quase não há conta para esconder a espera.

Trechos médios (1 000 a 100 000 pontos): a sobreposição aparece. Com 1 000 pontos a conta ($\sim 0,9 \mu\text{s}$) tem a mesma ordem da latência. Se não houvesse sobreposição, a v2 levaria cerca de 2,3 μs (comunicação + conta); ela leva 2,0 μs , porque parte da latência fica escondida atrás dos pontos internos. O ganho sobre a v1 cai de $\sim 1,9\times$ para $\sim 1,2$ – $1,3\times$ à medida que a conta passa a dominar, pois a latência (alguns μs) economizada vira uma fração cada vez menor do passo.

Trechos grandes (1 000 000 de pontos): a computação domina. Cada passo leva ~ 3 ms de conta contra poucos μs de comunicação; as três versões ficam empatadas dentro da variação entre execuções, já que não há quase nada para esconder.

v2 $\times$ v3. As duas têm desempenho parecido. Como as mensagens têm só 8 bytes, o OpenMPI as envia em modo *eager* e a rede progride sozinha enquanto o processo calcula; por isso o MPI_Wait depois da conta já encontra a comunicação quase pronta e o MPI_Test não tem o que adiantar. A v3 ainda paga o custo das chamadas a MPI_Test e do laço em blocos, o que a deixa um pouco mais lenta nos trechos grandes. O MPI_Test vale mais quando a biblioteca só avança a comunicação dentro das chamadas MPI (mensagens grandes, protocolo *rendezvous*).

Entre as repetições, os tempos variaram até ~50% nos trechos de 10 000 a 100 000 pontos (ruído de rede e de sincronização entre processos), por isso as diferenças pequenas nessa faixa não são conclusivas; os ganhos de ~2× nos trechos pequenos se repetiram em todas as execuções.

6. Conclusão

A comunicação não bloqueante reduziu o tempo por passo à metade quando a troca de bordas domina, ao permitir que as mensagens com os dois vizinhos fiquem em trânsito juntas, e escondeu parte da latência atrás da atualização dos pontos internos quando a conta tem a mesma ordem da latência. Quando a conta domina, as três versões se equivalem. Entre `MPI_wait` e `MPI_Test` não houve diferença relevante para mensagens pequenas, e a versão com `MPI_wait` é a mais simples.

7. Código-Fonte

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define K 0.25 /* alfa*dt/dx^2 (estável se <= 0,5) */
6 #define BLOCO 256 /* pontos atualizados entre dois MPI_Test */
7
8 /* Atualiza os pontos i = a .. b-1, */
9 static void atualiza(double *u, double *v, int a, int b) {
10     for (int i = a; i < b; i++)
11         v[i] = u[i] + K * (u[i - 1] - 2 * u[i] + u[i + 1]);
12 }
13
14 /* 1 quando todas as requisições terminaram. */
15 static int pronto(MPI_Request *req, int nr) {
16     int ok = 1, f;
17     for (int k = 0; k < nr; k++) {
18         MPI_Test(&req[k], &f, MPI_STATUS_IGNORE);
19         ok = ok && f;
20     }
21     return ok;
22 }
23
24 int main(int argc, char **argv) {
25     int rank, size;
26     MPI_Init(&argc, &argv);
27     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
28     MPI_Comm_size(MPI_COMM_WORLD, &size);
29
30     int versao = atoi(argv[1]), n = atoi(argv[2]), passos = atoi(argv[3]);
31     int esq = rank - 1, dir = rank + 1; /* vizinhos */
32
33     /* u[1..n]: trecho do processo; u[0] e u[n+1]: células extras (bordas) */
34     double *u = calloc(n + 2, sizeof(double));
35     double *v = calloc(n + 2, sizeof(double));
36     if (rank == 0) u[0] = v[0] = 100.0; /* ponta esquerda quente */
37
38     double t0 = MPI_Wtime();
39     for (int p = 0; p < passos; p++) {
40         if (versao == 1) {
41             /* pares enviam primeiro e ímpares recebem, depois invertem */
42             for (int f = 0; f < 2; f++) {
43                 if ((rank + f) % 2 == 0) {
44                     if (esq >= 0) MPI_Send(&u[1], 1, MPI_DOUBLE, esq, 0, MPI_COMM_WORLD);
45                     if (dir < size) MPI_Send(&u[n], 1, MPI_DOUBLE, dir, 0, MPI_COMM_WORLD);
46                 } else {
47                     if (esq >= 0) MPI_Recv(&u[0], 1, MPI_DOUBLE, esq, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
48                     if (dir < size) MPI_Recv(&u[n + 1], 1, MPI_DOUBLE, dir, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
49                 }
50             }
51             atualiza(u, v, 1, n + 1);
52         } else {
53             MPI_Request req[4];
54             int nr = 0;
55             if (esq >= 0) {
56                 MPI_Irecv(&u[0], 1, MPI_DOUBLE, esq, 0, MPI_COMM_WORLD, &req[nr++]);
57                 MPI_Isend(&u[1], 1, MPI_DOUBLE, esq, 0, MPI_COMM_WORLD, &req[nr++]);
58             }
59             if (dir < size) {
60                 MPI_Irecv(&u[n + 1], 1, MPI_DOUBLE, dir, 0, MPI_COMM_WORLD, &req[nr++]);
61                 MPI_Isend(&u[n], 1, MPI_DOUBLE, dir, 0, MPI_COMM_WORLD, &req[nr++]);
62             }
63             if (versao == 2) {
64                 atualiza(u, v, 2, n); /* interior não usa as bordas */
65                 for (int k = 0; k < nr; k++) MPI_Wait(&req[k], MPI_STATUS_IGNORE);
66             } else {
67                 int i = 2;
68                 while (!pronto(req, nr) && i < n) { /* um bloco por teste */
69                     int fim = i + BLOCO < n ? i + BLOCO : n;
70                     atualiza(u, v, i, fim);
71                     i = fim;
72                 }
73                 atualiza(u, v, i, n); /* restante do interior */
74                 while (!pronto(req, nr)) {} /* interior pronto: só espera */
75             }
76             atualiza(u, v, 1, 2); /* bordas, com as células extras */
77             atualiza(u, v, n, n + 1);
78         }
79         double *t = u; u = v; v = t;
80     }
81     double t = MPI_Wtime() - t0;
82
83     if (rank == 0) /* versao,processos,pontos por processo,passos,tempo,u[n] */
84         printf("%d,%d,%d,%d,%.6f,%.10f\n", versao, size, n, passos, t, u[n]);
85     free(u);
86     free(v);
87     MPI_Finalize();
88     return 0;
89 }
90
```

Figura 2 — calor.c.